Final Report: Fantasy Football Point Predictor

Team: Cedric Lambert, Austin Miller, and Ryan Pierce | *SIADS 696 012 FA 2025*

1. Introduction

As of 2022, 29.2 million people play fantasy football annually within the United States with fantasy sports as a whole having 50 million players and accounting for $11 billion in total revenue.[1] This game, so called, is played through the fantasy draft of a team of existing NFL players. Each week teams match up against each other to continue the mimicry of a real football team. The players' stats are collected and run against a scoring algorithm, which, after all the games for the week are played, the teams with the highest score win their matchup. This goes on until the season is over and a select number of teams get to play in a playoff series that ends in a championship.

On top of the significant amount of people and money in this, there is also a significant amount of fun to be had. Any "competitive edge" is sought out. Usually, this is found in following analysts and going to websites that have projections based on analyst predictions in order to pick which players should "start" and which should be "benched". This project seeks to add a more quantitative element to the process through predicting player scores based on past performance in order to gain the craved "competitive edge" in order to defeat all opposition.

The members of this project all have a mutual enjoyment for fantasy football and have been seeking to find that "competitive edge" using the realm of data science. Any benefit we get from this project correlates positively with the metrics we use to identify and quantify our joy and happiness during moments of self-reflection. So, it's a big deal.

We compared Linear Regression (LR), XGBoost (XGB), and Multi-Layer Perceptron (MLP) Regression models. Ultimately, we found that the MLP model performed the best over the metrics of mean absolute error (MAE), root mean squared error (RMSE), and r-squared. However, the LR model, which we used as a baseline, was marginally worse.

This idea of adding data science into the mix of tools for understanding what players will play the best is not new. However, our approach intends to utilize a new mixture of modelling techniques, datasets, and feature processing. Usually, the resources fantasy football players will use have to do with a ranking structure. Our project, on the other hand, seeks to predict the fantasy points for each player instead of simply ranking.

We also performed an analysis using unsupervised learning methods, both for dimensionality reduction and also analyzing clusters based on features to ascertain any insights that could be useful in any way with choosing who to start and who to bench. Our results found that Principal Component Analysis (PCA) successfully compressed correlated features while improving extreme outlier predictions. K-Means on the features yielded k=3 stable QB archetype clusters: pocket passer, dual threat, and game manager.

2. Related Work

This project is not an extension of our previous coursework, but comparable research exists and is outlined below.

1. ffdraft.app – Uses a different dataset and addresses a similar problem with a different approach. Ffdraft.app appears to be a fantasy football draft assistant that uses projections from ESPN, CBS, and NFL, and calculates dynamic Value Over Replacement (VOR) to help optimize roster decisions. That said, our approach is different from Ffdraft.app. For example, our use of supervised projections and clustering incorporates advanced analytics that extend beyond heuristic ranking. Additionally, our structured methodology outlines model types, features, limitations, and applications.
2. AI-Powered draft assistant – This project has a similar objective but takes a very different approach. This project consolidates AI-generated research from various LLMs from ChatGPT, Google

Gemini, Claude, and Perplexity. It then relies on this consolidated material as part of the queries used for draft-day suggestions. Our project is different as it uses model output to predict the best player based on historical stats.

3. Predicting Fantasy Football Points Using Machine Learning – Zhang et al.'s project also has a similar objective and approach to ours but uses only supervised prediction methods. They train/evaluate on historical NFL data (2015-2016) and use ML models: ridge, Bayesian ridge, elastic net, random forest, and gradient boosting. Our project is different and an improvement as we use both more current data, have reproducible workflows, and we use both supervised and unsupervised methods.

3. Data Source

- **Source:** Nflreadpy – nflreadpy is a Python library for interacting with NFL data sourced from nflfastR, nfldata, dynastyprocess, and Draft Scout.
- **Location:** https://nflreadpy.nflverse.com
- **Format:** pandas DataFrames (CSV/Parquet)
- **Key columns:** player_id, player_name, position, season, week, team, opponent_team, age, years_exp, fantasy_points, passing_yards, passing_tds, passing_interceptions, passing_2pt_conversions, sack_fumbles_lost, rushing_yards, rushing_tds, rushing_fumbles_lost, rushing_2pt_conversions, receiving_yards, receiving_tds, receiving_fumbles_lost, receiving_2pt_conversions
- **Shape:** 62,859 rows × 127 columns
- **Time coverage:** 2014 NFL season to current

4. Feature Engineering

Our data sources possessed the ability to pull complete historical NFL data in various formats dating back to the year 1999. We decided to use historical data from the last 10 NFL seasons, dating back to 2014. The formats included options such as play-by-play data, player game statistics, team game statistics, game schedules with results, player information, team rosters, advanced stats, depth charts, and much more. In order to create a dataset with the ideal features for our Fantasy Point prediction model, a number of dataset merges took place, specifically using team rosters, player game statistics, team game statistics, and game schedules with results. Merging these datasets together allowed us to have a robust feature set of week-by-week statistical outputs for each player that has played throughout the last 10 seasons.

Feature engineering for the project was performed under a fairly simple approach - by identifying all of the stat categories that have a direct impact on fantasy point outputs. To limit the scope of our project and features, it was decided that position groups under analysis would be limited to Quarterbacks(QB), Running Backs(RB), Wide Receivers(WR), and Tight Ends(TE), leaving out Kickers and Defense/Special-Teams. While each of the position groups here are typical of any fantasy football league, there are an assortment of fantasy football scoring systems that can be implemented in any given league. The scoring system we landed on is Standard scoring (non-PPR) - PPR stands for Points Per Reception, indicating that our chosen scoring system will not involve that aspect of fantasy football scoring.

With our target variable as 'fantasy_points', our dependent variables, based on the Standard scoring system and given our selected position groups, became easily identifiable - "passing_yards": 1/25, "passing_tds": 4.0, "passing_interceptions": -2.0, "passing_2pt_conversions": 2.0, "sack_fumbles_lost": -2.0, "rushing_yards": 1/10, "rushing_tds": 6.0, "rushing_fumbles_lost": -2.0, "rushing_2pt_conversions": 2.0, "receiving_yards": 1/10, "receiving_tds": 6.0, "receiving_fumbles_lost": -2.0, "receiving_2pt_conversions": 2.0. For reference, each value you see here is the number of fantasy points awarded for a given statistic. In addition to these statistical dependent variables, we also included demographic dependent variables 'age'(player age) and 'year_exp'(years of experience).

We decided to incorporate one last set of dependent variables related to the opposing defense that each player played against on a given week and, to stay true to the nature of fantasy points, we decided to use defense & special teams(D/ST) fantasy points. While this statistic was not readily available, we were able to calculate this figure for each opposing team using the various statistics that impact a D/ST's fantasy points including 'def_tds', 'special_teams_tds', 'points_allowed', and more. For each week of every season, the D/ST's fantasy points were also ranked from best to worst(1-32), providing us with our final 2 dependent variables 'fp_def_st' and 'fp_def_st_rank'.

To enhance our predictive model even further, rolling averages for each statistical dependent variable(including 'fp_def_st' and 'fp_def_st_rank') were produced in values of 3, 5, and 8, providing us with information for a given week's current game, last 3 games, last 5 games, and last 8 games respectively without barriers across seasons or missed games. The final element to our dataset to note is in regards to all of the statistic dependent variables being lagged by 1-game. To put it simply, this allows us to evaluate a player's current week fantasy point output based on how they performed up until that point with respect to the defense they're playing against.

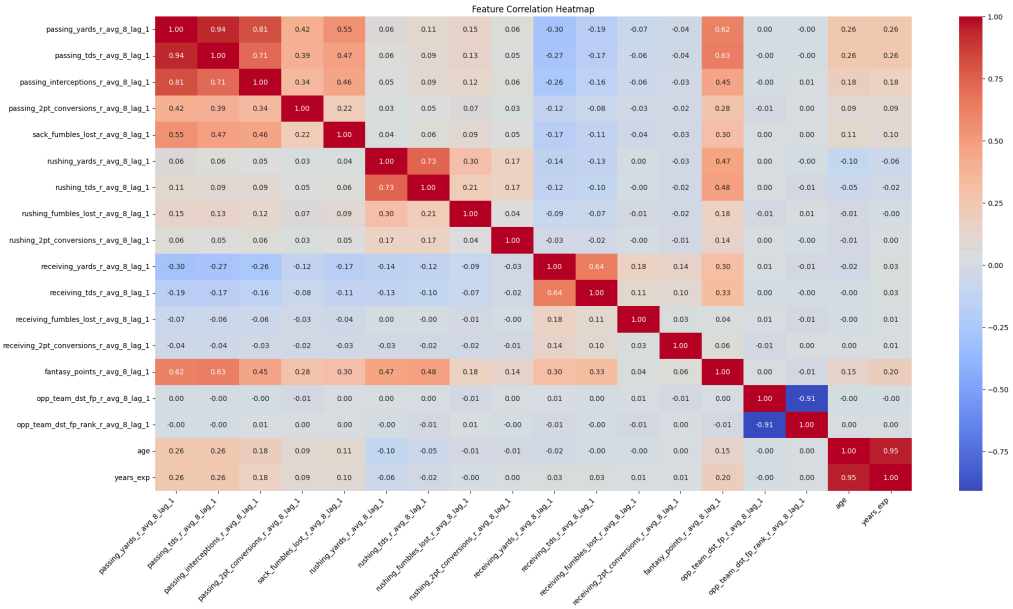


Figure 4.1 (Feature Correlation Heatmap with 1-Lagged 8-game Rolling Average Stats + Age & Years Exp)

For example, if Joe Burrow scored 30.1 fantasy points in Week 9 of the 2024 season, we're asking the question of how his recent performance across the specified spans(last 1, 3, 5, and 8 games) along with how the defense has performed(in terms of fantasy points across those same spans) impacts that output. His row of statistics for that week will display his passing yards from the last 1, 3, 5, and 8 games, his rushing yards across the same spans, his passing touchdowns, the number of fantasy points produced by the D/ST he's playing against, etc. Ultimately, the final 1-week lagged dataset we're performing all of our work based on has a shape of 60969 rows × 78 columns(66 of which are the dependent variable feature set).

5. Supervised Learning

5.1 Methods Description

Three supervised learning techniques were used with the goal of finding one that performs the best to use as a final model, and two feature representations were used.

Feature representations

1. Transformation through normalization was performed on the entire dataset using min max scaling. The dataset was normalized in order to avoid having certain attributes give more weight to the model due to their higher numeric values. One may think of how a quarterback might have 300 yards, 2 interceptions, and 1 touchdown. The min max scaling algorithm is relatively simple, and it converts each value within each attribute to be between 0 and 1. Sklearn's MinMaxScaler package was utilized to do this.
2. Principal Component Analysis (PCA) was performed on the entire dataset for dimensionality reduction. PCA also automatically normalizes the data so that the issue of one column having greater values does not impact the weight of the model as mentioned above. On top of this, PCA combines columns together based on how correlated they are in order to create a dataset of columns that are uncorrelated (orthogonal). With a goal of retaining 95% of the variance of the dataset when generating the components, we were able to reduce the number of input (also referred to as X) columns from 66 to 25 with the goal of increasing training speed with minimal sacrifice to performance when training models on it.

Supervised learning methods
1. Linear Regression (LR) was used as a baseline model of sorts, because it is fast to train and not particularly complicated compared to the other models that were used. Many times a LR performs comparably to a more complex model, which is why this method is still used frequently.
2. Extreme Gradient Boosting (XGBoost) was selected as an ensemble learning method that builds sequential decision trees to correct errors from previous iterations. XGBoost is particularly well-suited for this task because it handles complex feature interactions effectively and is robust to overfitting through its regularization parameters. The parameters tuned for this model included: learning rate, maximum tree depth, number of estimators, subsample ratio, and column sampling by tree. Default settings were maintained for other hyperparameters such as the objective function (reg:squarederror for regression) and the tree booster method.
3. Multi-Layer Perceptron Regressor (MLP) was also used as a model that can help find potentially non-linear relationships between offensive players and the defense they are playing against in a given week. The parameters used in tuning were: hidden layer sizes (we stuck with 2 hidden layers), maximum iterations, learning rate, and activation functions. Everything else was kept consistent with what is normal for the implementation of MLPs such as using the Adam solver and Mean Squared Error as the loss function.

Originally, the goal was to use sklearn's MLPRegressor package, but the performance was too slow for the number of models being generated and evaluated. On top of this, there was no way to speed it up since sklearn cannot use a GPU. Instead, a mimicked version of the MLPRegressor was built that utilized pytorch because pytorch can use a GPU. The skeleton of the code was found on the pytorch discussion forum and tweaked to suit our purposes.[2]

Tuning the MLP on pytorch was still very resource intensive and required using a GPU on a HPC environment for days. The parameters that were tuned and compared were as follows:
- Hidden Layer Sizes: 100-100, 500-500, 1000-1000
- Maximum Iterations: 100, 500, 1000
- Learning Rates: 0.01, 0.001, 0.0001
- Activation Functions: Tanh, ReLu

5.2 Supervised Evaluation
In evaluating performance of the models decided on, three major metrics stood out to help in the assessment.
1. Mean Absolute Error (MAE) takes the absolute value of errors both positive and negative. So, an error of 1 could mean the model tends to before either positively by 1 more fantasy point or negatively by 1 fantasy point. This metric is very intuitive.
2. Root Mean Squared Error (RMSE) squares the error, takes the mean of the new value, and then takes the square root of the mean value. The meaning of the value is similar to the mean absolute error: If the model is off by 1 then it is off by 1 in either direction. What makes RMSE helpful is that it is more sensitive to outliers, which would cause it to give a larger number than MAE if there are outliers.

3. R-Squared is a metric that is usually between 0 and 1. It represents how strong of a relationship there is between the target (or Y variable) and the input (X) variables. This relationship is described in terms of the variance within the target that the model (the inputs) explains. Another term that is used to describe R-Squared is goodness of fit.

The best performing model within the three groups was the MLP (see Figure 5.2.1 below). The second best performing model was the LR. Using the MAE as the most intuitive metric will show that all of the models were only off from each other by less than a single rushing/receiving year or less than 2.5 passing yards since the MAE tells us that the differences between the models themselves were roughly 0.1 fantasy points (see the standard scoring description above), and the models were all off by a little under 4 fantasy points in either direction in their predictions on player performance.

Model	Feature Rep.	MAE	RMSE	R-Squared
Linear Regression	MinMax	3.873 ± 0.044	5.381 ± 0.058	0.406 ± 0.002
Linear Regression	PCA	3.906 ± 0.044	5.414 ± 0.057	0.398 ± 0.003
XGBoost	MinMax	3.951 ± 0.048	5.394 ± 0.068	0.403 ± 0.003
XGBoost	PCA	3.884 ± 0.046	5.395 ± 0.061	0.402 ± 0.003
MLP Regressor	**MinMax**	**3.826 ± 0.079**	**5.37 ± 0.06**	**0.408 ± 0.003**
MLP Regressor	PCA	3.955 ± 0.109	5.559 ± 0.278	0.363 ± 0.076

Figure 5.2.1 (These numbers were calculated using the mean of the error metric on 5 fold cross validation)

The best performing MLP model on the normalized data ended up being the one with the smallest size of hidden layers (100-100), smallest maximum iterations (100), and the smallest learning rate (0.0001) while using the Tanh activation function. *The maximum iterations being relatively small may have acted to stop the model early before it fully converged, which increased generalizability.*

Sensitivity Analysis

Figure 5.2.1 shows the loss between the train and validation of the best performing model while Figure 5.2.2 shows the same but with a maximum iterations of 1000 (notice the difference in x-axes in 5.2.1 and 5.2.2). At a maximum iterations of 100, the training and validation lines stay very close together once they meet around iteration 10. At a maximum iterations of 1000, the validation line ends up significantly higher than the training line which indicates overfitting.

Meanwhile, increasing the size of the hidden layers, while holding everything else constant, appears to have a similar effect. In Figure 5.2.3, the validation loss is higher than the training loss compared to Figure 5.2.1.

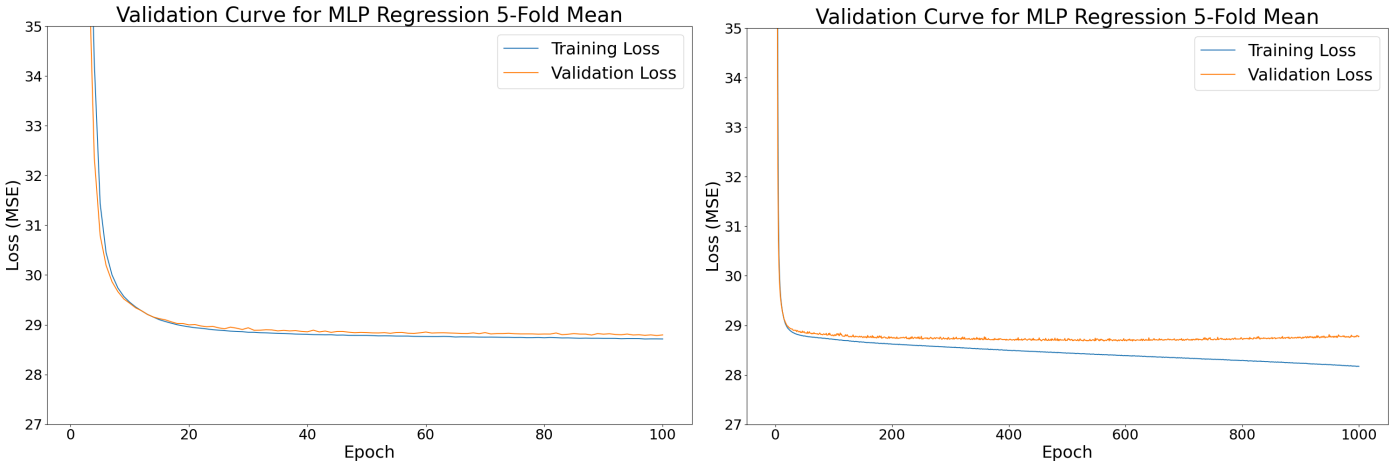


Figure 5.2.1 (Hidden layer size 100,100. Maximum iterations 100. Learning rate 0.0001) [right] and Figure 5.2.2 (Hidden layer size 100,100. Maximum iterations 1000. Learning rate 0.0001) [right]

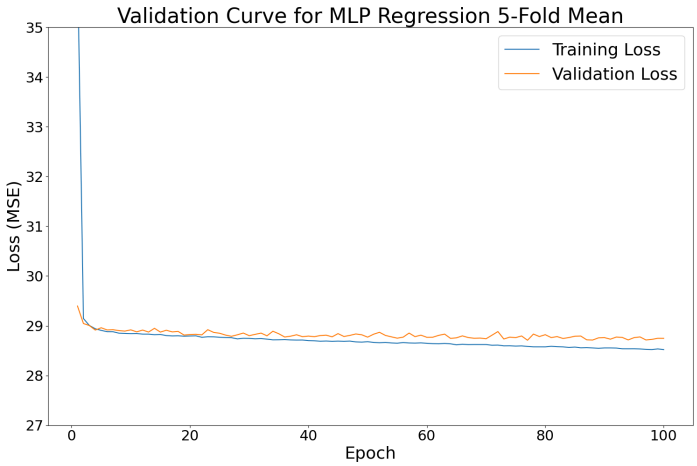


Figure 5.2.3 (Hidden layer size 1000,1000. Maximum iterations 100. Learning rate 0.0001)

Feature Analysis

A feature permutation analysis was performed in order to analyze feature importance. The 5 most important features can be seen in Figure 5.2.4, and the 5 least important features can be seen in Figure 5.2.5.

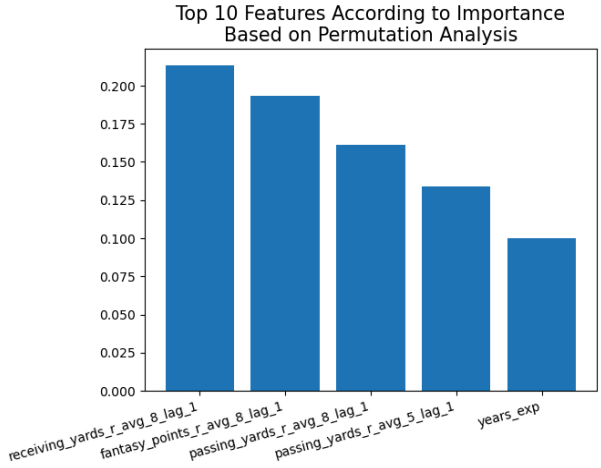


Figure 5.2.4

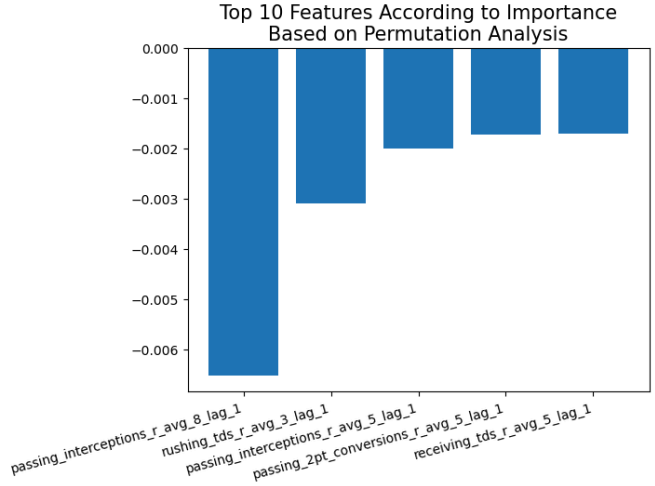


Figure 5.2.5

We found that receiving yards over the last 8 games was the most important feature in the entire dataset. This is likely influenced in part because the proportion of wide receivers and tight ends (the two positions that rely heavily on receiving yards) is 0.608. More interestingly, the number of passing interceptions over the last 8 games is the lowest feature in importance. This is likely due to the quarterback making up only 10% of the entire dataset and the number of interceptions thrown being a numerically small metric (quarterbacks do not throw more than a few per game and many times they throw zero).

Ablation Analysis
Ablation analysis was also performed to investigate the importance of the lag periods.

Lag Period or Baseline	Mean Absolute Error
Week Rolling Average	3.681215
1 Week Rolling Average	3.688057
5 Week Rolling Average	3.726455
Baseline	3.812106
8 Week Rolling Average	**4.769469**

Figure 5.2.6

Without the 8 week rolling average (see Figure 5.2.6), the mean absolute error increases significantly which indicates that the 8 week rolling average has a significant effect on the model's predictability. However, when the other 3 periods are removed independently, the mean absolute error decreases for them. This indicates that there is noise that is introduced with these periods.

Tradeoffs
There were multiple tradeoffs that had to be considered in evaluating and choosing the best model. Ultimately, the MLP's performance against the metrics was why it was chosen, but it was the longest to perform hyper-parameter tuning on due to the dimensionality of the data. To contrast, the linear regression was the fastest to train, but it contained a greater error window.

A greater accuracy for the models may have been gained through training position based models. Instead of a single model for all the players, we could have broken them down by QB, RB, and Receiving (WR and TE). A model that specifically is focusing on players that, for example, primarily throw the football may uncover more specific patterns that a model trying to understand the whole set of position groups may not.

Failure Analysis
Limitations existed in our data and models that were found during analysis. First, as said above, our dataset contained rolling averages for players across seasons, but since rookies did not have any NFL history, their first games were not recorded in the dataset (it is a *systematic error* since there are no data for this type of record). If a record for rookies was manually generated and inserted into the MLP, it yielded a prediction between approximately 0 and 2 fantasy points. This is arbitrary and there was no way for the model to predict a player like the rookie QB C.J. Stroud scoring 10.7 points in his first game (season: 2023, week: 1).[3]

One way to fix this would be to include a rookie flag column (a "yes" or "no" binary value to indicate if a player is a rookie during a particular season) and include some indicators for how good the rookie is

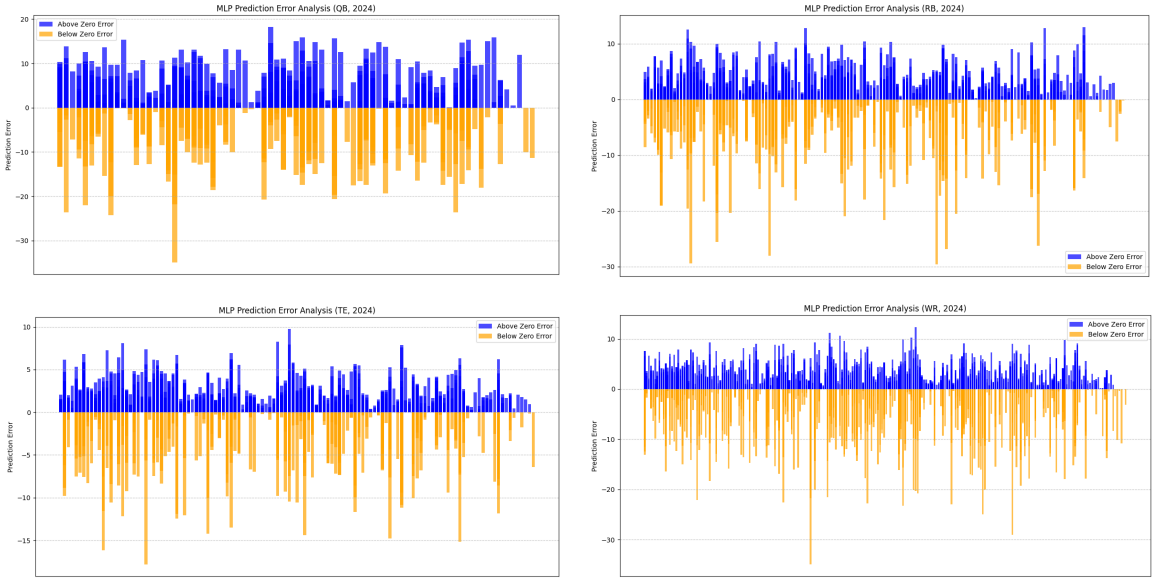
supposed to be. This could be a sentiment analysis from analysts, historical college stats, combine/draft stats, or a combination of the three.

Second, the model had no entries for games that a player missed due to injury (it is an *operational error* since there was a flaw in the dataset caused by the way we engineered the features). This skewed the rolling averages and caused the model to interpret the players to pick up where they left off in their scoring. For example, De'Von Achane scored 20.5 points before being injured and missing 5 games in 2023. The model predicted that he would score 16.2 points, but instead he scored 0.5.[3]

A fix for this would be to include injured records and add an injury flag to indicate that a player has both scored 0 points and it is because he is injured. That way the model would hopefully learn a good, generalizable pattern that accounts for how players play when coming off of an injury.

Finally, there are the random errors found in players having games where they score 30+ points and all the games before then had significantly smaller totals. For example, Derrick Henry in season 2018, week 14 scored 47.8 points after having only 3 games that crossed 10 points and 1 game that crossed 15 points before that in 2018.[3]

Figure 5.2.7-10 shows the error predictions at the position level, and the overwhelming size of negative errors (predicted points was too small) compared to positive errors (predicted points was too big) helps illustrate the big games all players had.



Figures 5.2.7, 5.2.8, 5.2.9, & 5.2.10 (MLP Prediction Error Bar Charts)

These are *random errors* and do not show a clear pattern. One possible solution would be to train a model that calculates a big game likelihood score that could then be inserted as an attribute into the overall dataset for the MLP model. That way we could capture some of the perceived noise if there are any underlying trends that a machine learning model could predict.

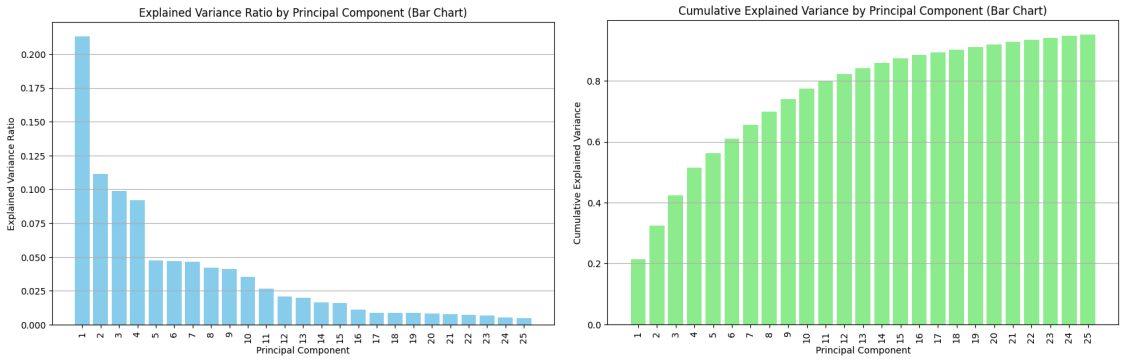
6. Unsupervised Learning

6.1 Methods Description

PCA

The Principal Component Analysis performed on the final 1-week lagged dataset successfully reduced the feature space while retaining 95% of the variance. Feature scaling was highly critical as fantasy

football statistics vary dramatically in magnitude (touchdowns vs. yards), and PCA is highly sensitive to scale. We standardized all features before applying PCA to ensure each feature contributed appropriately to the principal components. The resulting 25 Principal Components provide a logical and efficient dimensionality reduction of the dataset features.



Figures 6.1.1(PCA Explained Variance Ration) & 6.1.2(PCA Cumulative Explained Variance)

K-Means Clustering

We used K-Means clustering because it is simple, fast, and produces easy-to-explain group centers/clusters. The K-means clustering analysis was performed on the final 1-week lagged dataset with 8-game rolling averages. We focused on quarterbacks (they drive weekly fantasy scoring and show relatively lower variance [4]) and aimed to cluster by archetype. To capture style, we created a rush share column 'rushing_yards_r_avg_8_lag_1'/(['passing_yards_r_avg_8_lag_1']+['rushing_yards_r_avg_8_lag_1']) for clustering to capture “pocket vs. dual-threat vs. game-manager” styles. To keep seasons comparable, we standardized each season using StandardScaler (z-score) and removed extreme outliers keeping values between the 1st and 99th percentiles. We also excluded fantasy points to avoid leakage.

After creating rush_share, we examined its distribution and found it to be right-skewed (Figure 6.1.2: rush_share_8 Frequency Distribution). To address this skew, we log-transformed rush_share and paired it with the 8-week average passing yards feature for clustering.

We completed hyperparameter (k) selection using a combination of the Silhouette score, the elbow method (inertia), and the Calinski–Harabasz (CH) Index. An ideal k would be one that is the smallest value that showed a clear elbow, a local peak/plateau in Silhouette, and a high CH. Our results are shown in Figure 6.1.3 below.

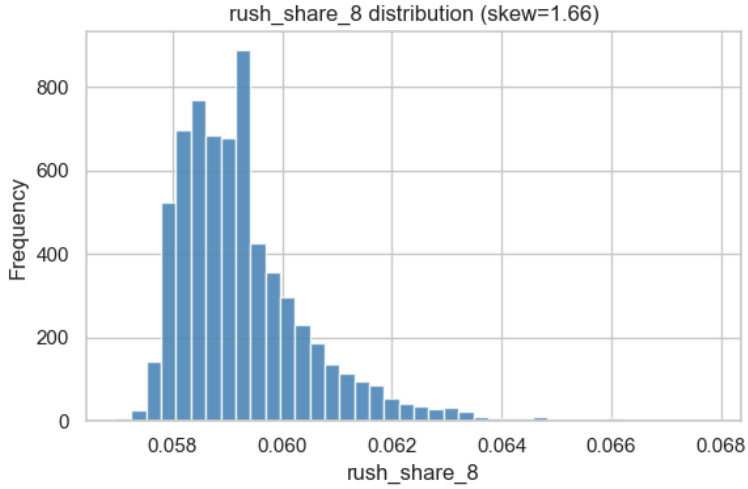


Figure 6.1.2 (rush_share_8 Frequency Distribution)

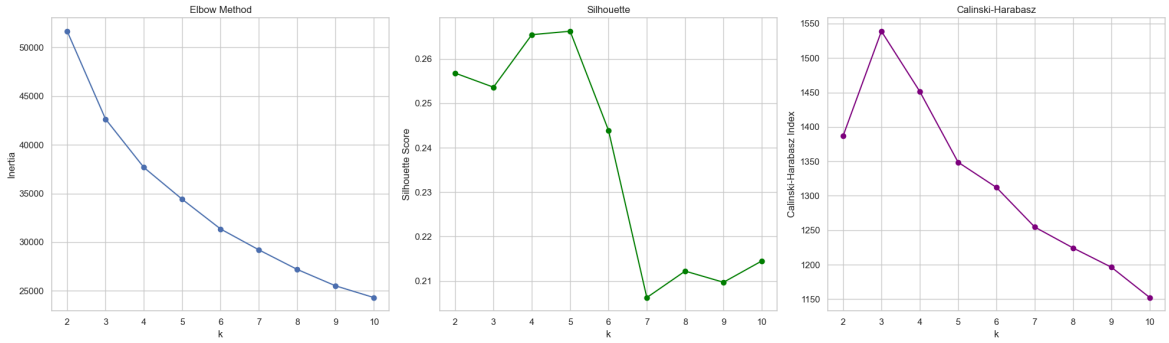


Figure 6.1.3 (Elbow v Silhouette v Calinski-Harabasz)

Since there is no clear elbow in the “Elbow Method” graph we focused on getting the lowest k (highest inertia) possible. Silhouette indicates strong separation for k = 2–5, and CH peaks at k = 3. Balancing quality and simplicity, we selected k = 3 for clustering.

6.2 Unsupervised Evaluation

PCA

Given the creation of rolling averages for all statistic dependent variables in our original dataset, which generated 4 features per statistic, PCA successfully captured the rolling average patterns and feature correlations in a programmatic fashion. This is evident in the feature contributions displayed in figures 6.1.3 and 6.1.4. In certain instances, the top 4 contributing features to a Principal Component consist of the 4 rolling averages from a single statistic. For example, PC1's top 4 features are the rolling averages for passing_yards, while PC5's top 4 features are receiving_2pt_conversions. In other instances, the top 4 features share direct correlations based on their naming conventions. For example, PC2's top 4 features include rushing_yards and rushing_tds, while PC3 includes receiving_yards and receiving_tds.

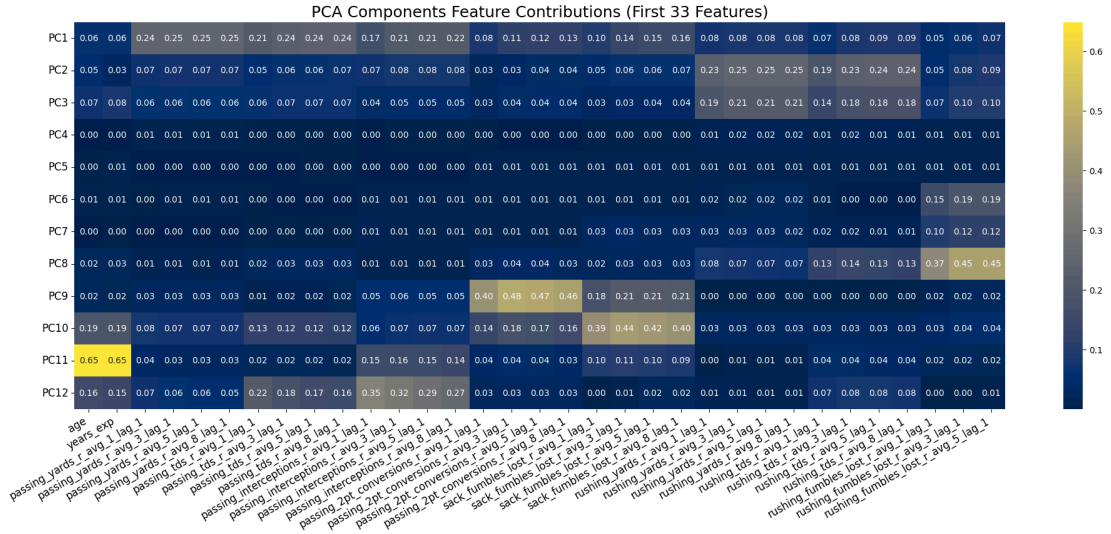


Figure 6.2.1(Top 12 PCA Components Feature Contributions - First 33 Features)

The contributing features within each Principal Component enable meaningful renaming based on their top contributors. For instance, PC1 becomes PC_passing_1, PC2 becomes PC_rushing_1, and so on. This approach preserves not only the variance from the original dataset but also the interpretability of the dependent variables. The compressed feature set, reduced from 60,969 rows × 66 columns to 60,969 rows × 25 columns, enables significantly faster model training while maintaining comparable accuracy due to the retained variance. In practical applications, the speed-accuracy tradeoff proves worthwhile, as

the Supervised Learning Regression models using PCA consistently outperformed their counterparts. Most importantly, even the poorest-performing PCA model (MLP Regressor) achieved 97% of the performance (measured by MAE) compared to its non-PCA equivalent.

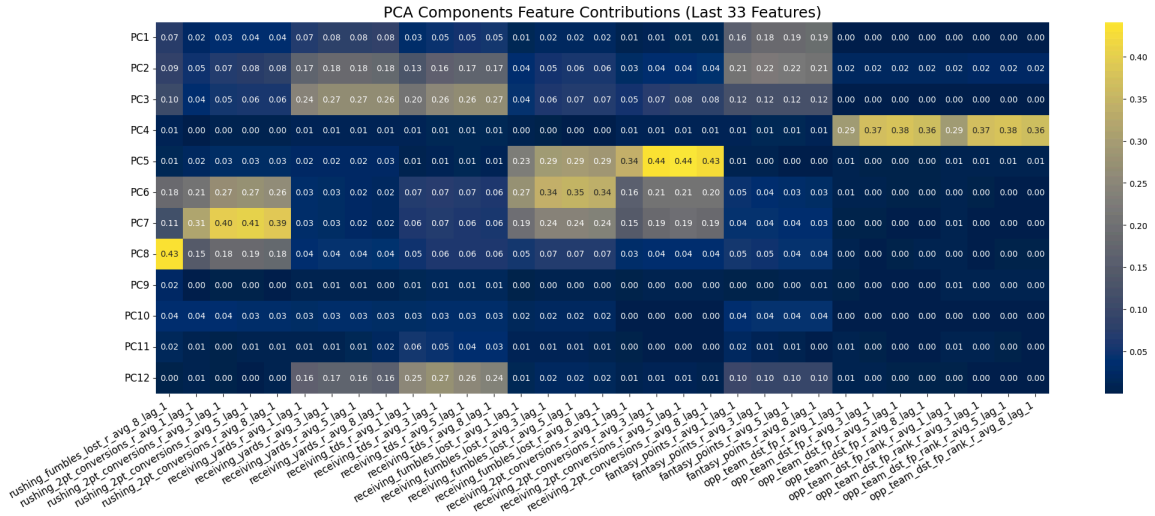


Figure 6.2.2 (Top 12 PCA Components Feature Contributions - Last 33 Features)

K-Means Clustering

The evaluation metrics we chose were the Elbow Method (Intertia) for compactness, Silhouette Score for separation, and the Calinski–Harabasz (CH) Index for variance (Figure 6.2.3, Elbow v Silhouette v Calinski-Harabasz). These metrics combined provide a defensible k. We sanity-checked the k choice with the traditional k-means scatterplot view (Figure 6.2.4, QB Archetypes - rush_share vs passing yards) to confirm the clusters were interpretable.

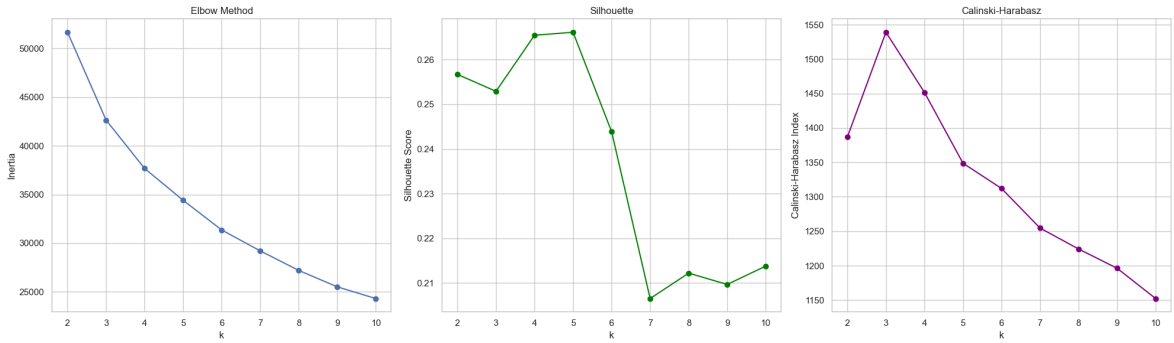


Figure 6.2.3 (Elbow v Silhouette v Calinski-Harabasz)

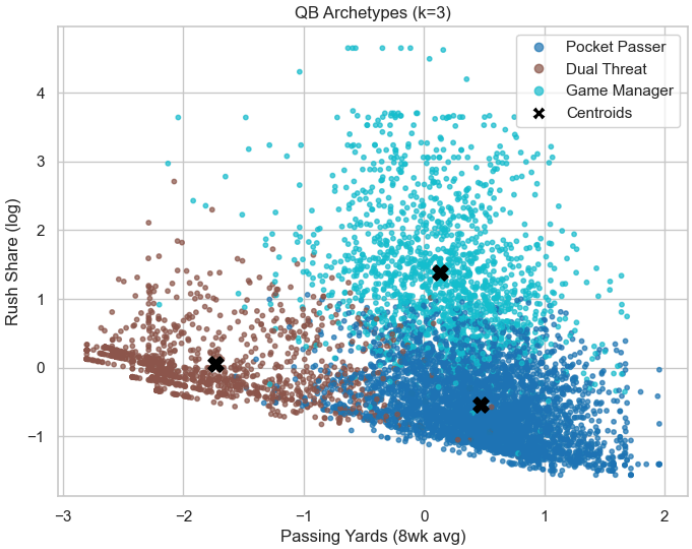


Figure 6.2.4 (QB Archetypes - rush_share v Passing Yards)

We confirmed our k choice with a sensitivity analysis over five random starts and k = 2–10. k = 3 was stable across seeds, with near-zero standard deviation for inertia, Silhouette, and CH (Figure 6.2.5: Std_dev - Elbow v Silhouette v Calinski-Harabasz).

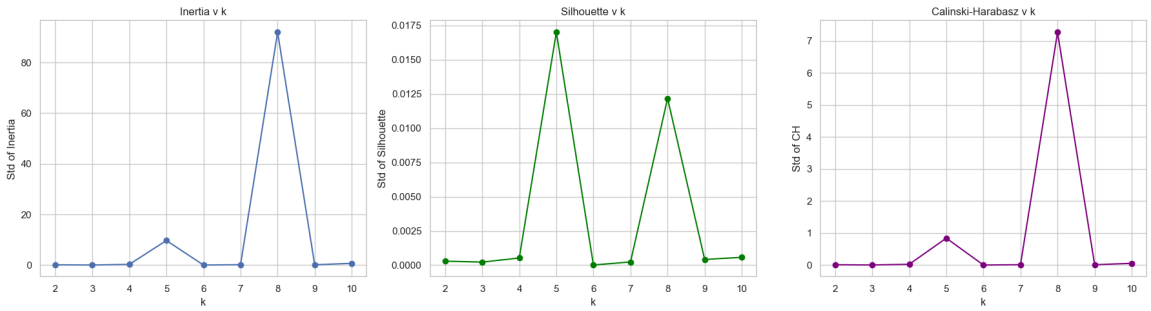


Figure 6.2.5 (Std_dev - Elbow v Silhouette v Calinski-Harabasz)

7. Discussion

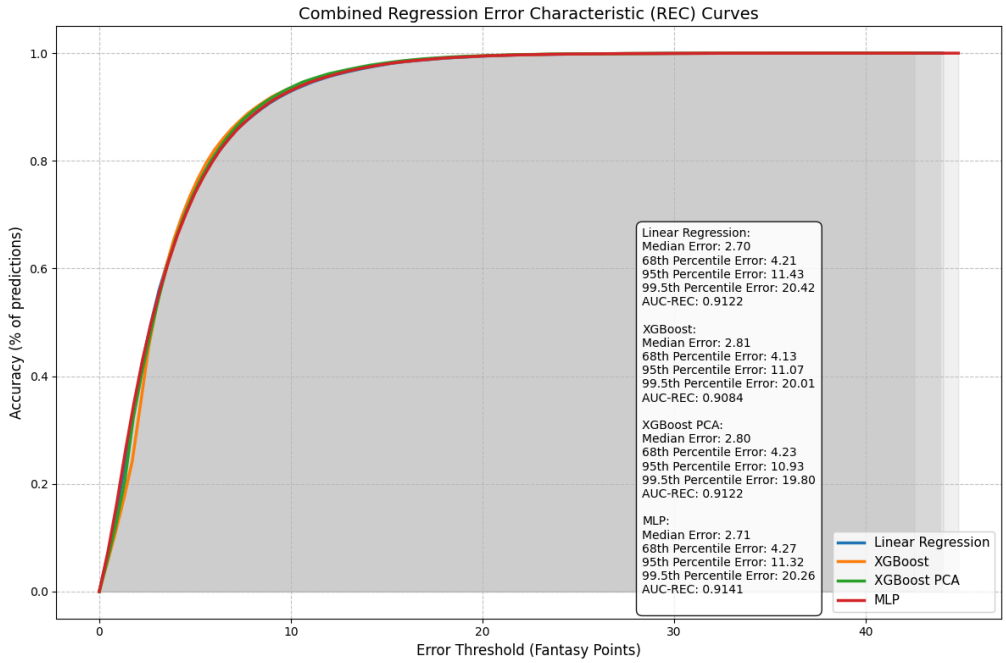


Figure 7.1 (Combined Regression Error Characteristic (REC) Curve)

7.1 What we learned from Supervised Learning

Linear Regression, XGBoost, and MLP

Since we already know what to expect in overall model performance given the previous MAE, R2, and RMSE comparisons, the most interesting findings using Regression Error Characteristic (REC) Curves emerge from analysis of the percentile errors. While the AUC-REC values(see Figure 7.1) confirmed our

expectations (MLP at 0.9141, Linear Regression at 0.9122, XGBosst at 0.9121), the distribution of errors across percentiles revealed surprising patterns about where each model excels.

XGBoost demonstrated the strongest performance at extreme outliers, achieving the best 99.5th percentile error of 20.01 fantasy points compared to the MLP's 20.26 and Linear Regression's 20.42. It also performed best at the 95th percentile (11.07 vs 11.32 and 11.43) and 68th percentile (4.13 vs 4.27 and 4.21). This suggests XGBoost's ensemble of decision trees effectively handle the most difficult predictions across the entire error distribution, excelling particularly at capturing extreme performances.

However, XGBoost had the worst median error at 2.81 compared to the MLP's 2.71 and Linear Regression's 2.70. This tradeoff is revealing: XGBoost appears to prioritize accuracy on challenging, high-variance predictions over optimizing performance on the most typical cases where all models already perform well. The difference at the median is small (0.1 fantasy points), but consistent with XGBoost's design philosophy of aggressively fitting difficult examples through boosting.

The MLP occupied a middle ground, with the best overall AUC-REC but mixed percentile performance. It outperformed Linear Regression at extreme tails but couldn't match XGBoost's consistency across difficult predictions, suggesting that while the neural network learns useful representations, the tree-based ensemble more effectively balances performance across the full range of prediction difficulty. The near-convergence of all three models reinforces the conclusion that fantasy point prediction may be a fundamentally linear problem once proper feature engineering is applied.

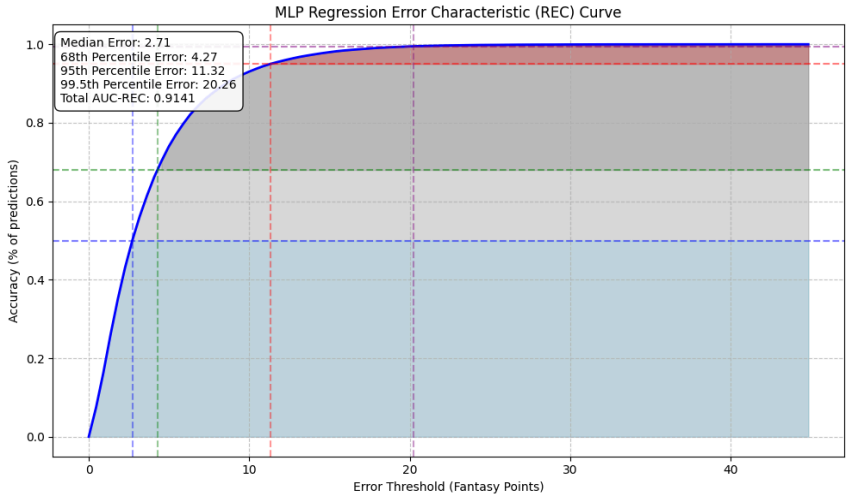


Figure 7.1.1 (MLP Regression Error Characteristic Curve) showing percentile shaded regions.

The primary challenge with Linear Regression was feature selection and engineering. Since linear models cannot automatically discover interactions or non-linear relationships, we wanted to ensure that our features adequately represented the prediction task at hand. As mentioned earlier, we addressed this by carefully constructing rolling averages that capture recent performance trends.

The primary challenge across both XGBoost and MLP was hyperparameter tuning for these more complex models. XGBoost required tuning parameters like learning rate, max depth, min_child_weight, subsample ratio, and regularization terms, while the MLP involved searching across layer sizes, number of layers, learning rates, dropout rates, and activation functions. The search spaces for both models were vast compared to Linear Regression's simplicity. We addressed this through systematic approaches: grid search with cross-validation for XGBoost and a hierarchical tuning strategy with cross-validation for the MLP that prioritized impactful parameters (learning rate and network depth). Balancing model complexity against the risk of overfitting was crucial for both models. The fact that both achieved only marginal

improvements over Linear Regression suggests we successfully avoided overfitting through appropriate regularization, but it also raises questions about whether the added complexity was justified given the minimal performance gains.

With more time, we would conduct a thorough feature interaction analysis to understand whether meaningful non-linear patterns exist that XGBoost and MLP aren't capturing. Since we intentionally limited our feature set to only those directly related to fantasy points in the standard scoring system, exploring interactions between our existing features could be valuable. For example, creating combined features like "touchdowns multiplied by rolling average fantasy points" or "total yards squared" might reveal important underlying patterns. If adding these explicit interaction terms improves Linear Regression substantially, it would confirm that non-linear patterns exist but need to be manually engineered rather than automatically discovered by XGBoost's tree-based splitting or learned by the MLP's hidden layers.

7.2 What we learned from Unsupervised Learning

PCA
The best Supervised Learning model with PCA turned out to be XGBoost. We found that there was a minimal difference in performance moving from pure XGBoost to XGBoost with PCA. Despite dimension reduction, the AUC-REC remained virtually identical (0.9122 with PCA vs 0.9121 without), and percentile errors stayed remarkably close. While we expected some redundancy given the rolling averages in our feature space, PCA's ability to preserve XGBoost's strengths was notable. The most surprising findings were at extreme outliers: XGBoost with PCA achieved a 99.5th percentile error of 19.80 compared to 20.01 for pure XGBoost. PCA even slightly improved the 95th percentile (10.93 vs 11.07), suggesting dimension reduction reduced noise affecting difficult predictions.

Interestingly, pure XGBoost performed better at the 68th percentile (4.13 vs 4.23 with PCA), but median errors were virtually identical (2.80 with PCA vs 2.81 without). This pattern reveals that PCA successfully compressed correlated rolling features while improving extreme outlier predictions, likely by reducing multicollinearity that destabilizes tree splits in edge cases. The slight degradation at the 68th percentile suggests that some information useful for moderately difficult predictions was lost in dimension reduction, though the overall impact remained minimal across the error distribution.

The main PCA challenge was determining how many principal components to retain. Too few would lose critical information, while too many would defeat the purpose of dimensionality reduction. We addressed this by analyzing the explained variance ratio and selecting components that captured sufficient variance while still meaningfully reducing dimensions.

With more time, position-specific PCA would be valuable as quarterbacks, running backs, and receivers have fundamentally different statistical profiles. Separate transformations for each position might capture variance more effectively than a single global PCA.

K-Means Clustering
We were surprised that the most informative clustering was not driven by raw stats, like TDs or INTs, but by style. After adding rush share and standardizing by season, quarterbacks separated into clusters we identified as pocket passers, dual threats, and a lower volume/game manager group.

A challenge was the lack of a clear elbow since the inertia curve was smooth. We combined the elbow method, Silhouette, and Calinski Harabasz to choose k, and selected the smallest k with strong Silhouette and CH, which pointed to k=3. If we had more time we would extend this archetype clustering to the remainder of the positions using other stats/transformations.

8. Ethical Considerations

8.1 Supervised Learning

We recognize that supervised predictions can be misinterpreted as guarantees. If users apply our outputs to sports betting or paid fantasy football leagues, they may incur financial losses. This tool is for informational and educational purposes only. It is not betting advice, does not guarantee outcomes, and may be wrong.

We can mitigate this by prominently displaying non-advisory disclaimers and using cautious, non-directive language.

8.2 Unsupervised Learning

Unsupervised clustering players can result in stereotype formation happening when identifying individual records. For instance, there may be a pull to stereotype that all QBs that have a higher tendency to get a lot of rushing yards as a certain ethnicity. This could result in bias towards expectations of player performance if the QB from a certain ethnicity gets less rushing yards, which could lead to vitriol on social media for that QB.

We can mitigate this by attaching an implicit bias reminder if these results are ever productionized and utilized in a live setting.

9. Statement of Work

All members collaborated on the data aggregation, manipulation, cleaning, and normalization.

Member:	**Ryan Pierce**	**Cedric Lambert**	**Austin Miller**
Supervised:	Regression	XGBoost	MLP, Regression
Unsupervised:	K-Means	PCA	
Visualizations:		REC Curve, Hyperparameters	Learning Rate, Feature Importance, Ablation
Admin:	Github Repo		

Appendix A - References

1. Fantasy Football financials — NFL's fan engagement engine https://medium.com/the-ao/fantasy-football-nfls-fan-engagement-engine-c713699859e6
2. Here is the source link for where the code was found that helped build the MLP Regressor: https://discuss.pytorch.org/t/sklearn-model-to-pytorch-model/33532
3. See the mlp_data_analysis notebook for the code on the rookie, injury, and big game failure analysis: https://github.com/ryanapierce/fantasy-football-assistant/blob/main/model_analysis/mlp_data_analysis.ipynb
4. Most Important Positions in Fantasy Football: https://www.weekonefantasy.com/most-important-positions-in-fantasy-football-7638968/
5. scikit-learn K-Means User Guide: https://scikit-learn.org/stable/modules/clustering.html#k-means
6. Silhouette: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.silhouette_score.html
7. Calinski–Harabasz Index: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.calinski_harabasz_score.html
8. Selecting k with silhouette analysis (example): https://scikit-learn.org/stable/auto_examples/cluster/plot_kmeans_silhouette_analysis.html
9. Interpreting and Validating Clustering Results with K-Means: https://medium.com/@a.cervantes2012/interpreting-and-validating-clustering-results-with-k-means-e98227183a4d
10. Skewness Be Gone: Transformative Tricks for Data Scientists: https://machinelearningmastery.com/skewness-be-gone-transformative-tricks-for-data-scientists/

Appendix B - Data Schema

Informational and Demographic Data Schema

Column Name	Description	Data Type
player_id	Unique identifier for each player	object
player_name	Full name of the player	object
player_display_name	Display name of the player	object
position	Player's position (QB, RB, WR, TE, etc.)	object
position_group	Grouping category for positions	object
headshot_url	URL link to player's headshot image	object
season	NFL season year	int32
week	Week number in the season	int32
season_type	Type of season (regular, playoff, etc.)	object
team	Player's team abbreviation	object
opponent_team	Opposing team abbreviation	object
age	Player's age	int64
years_exp	Years of professional experience	float64

Statistical Data Schema(1,3,5, & 8 Game Rolling Averages & 1-Game Lag Applied)

Column Name	Description	Data Type
passing_yards	Total passing yards	int32
passing_tds	Total passing touchdowns	int32
passing_interceptions	Total passing interceptions thrown	int32
passing_2pt_conversions	Successful 2-point conversions by passing	int32
sack_fumbles_lost	Fumbles lost due to sacks	int32
rushing_yards	Total rushing yards	int32
rushing_tds	Total rushing touchdowns	int32
rushing_fumbles_lost	Fumbles lost while rushing	int32
rushing_2pt_conversions	Successful 2-point conversions by rushing	int32
receiving_yards	Total receiving yards	int32
receiving_tds	Total receiving touchdowns	int32
receiving_fumbles_lost	Fumbles lost while receiving	int32
receiving_2pt_conversions	Successful 2-point conversions by receiving	int32
fantasy_points	Total fantasy points scored	float64
opp_team_dst_fp	Opponent team's defense/special teams fantasy points	float64
opp_team_dst_fp_rank	Ranking of opponent's defense fantasy points	float64

Appendix C - Notebook Catalog

Notebook names and their functionalities

Notebook Category	Notebook Description	Notebook Name
Dataset Generators	Generates main dataset	init_1_lag_dataset_generator.ipynb
Dataset Generators	Generates 5 fold cross validation datasets using MinMaxScaler	kfolds_dataset_generator.ipynb
Dataset Generators	Generates 5 fold cross validation PCA datasets	pca_dataset_generator.ipynb
Model Generators	Generates XGBoost regression model against 5 fold cv datasets	xgb_model_generator.ipynb
Model Generators	Generates linear regression models against 5 fold cv datasets	lr_model_generator.ipynb
Model Generators	Generates MLP Regressor models against 5 fold cv datasets	mlp_model_generator_pytorch.ipynb
Model Analysis	Failure analysis on MLP	mlp_data_analysis.ipynb
Model Analysis	Ablation analysis on MLP	mlp_model_ablation_analysis _features.ipynb
Model Analysis	Learning curve analysis on MLP	mlp_model_analysis.ipynb
Model Analysis	Feature importance on MLP	mlp_model_permutation_importance.ipynb
Model Analysis	Unused notebook that performed SHAP analysis on MLP	mlp_model_shap.ipynb
Model Analysis	K-Means Cluster Analysis	k_means_analysis.ipynb
Model Analysis	XGBoost model analysis and evaluation	xgboost_analysis.ipynb
Model Analysis	Principal Component Analysis (PCA) dimensionality reduction	pca_analysis.ipynb
Model Analysis	XGBoost model visualizations with PCA features	xgboost_pca_model_visuals
Model Analysis	Combined analysis comparing multiple models	combined_model_analysis
Model Analysis	Additional MLP model visuals	mlp_model_visuals
Model Analysis	Linear Regression model analysis	lr_analysis